
Problem 1

Refer to the Prostate cancer data set in Homework 11. Serum prostate-specific antigen (PSA) was determined in 97 men with advanced prostate cancer. PSA is a well-established screening test for prostate cancer and the oncologists wanted to examine the correlation between level of PSA and a number of clinical measures for men who were about to undergo radical prostatectomy. The measures are cancer volume, prostate weight, patient age, the amount of benign prostatic hyperplasia, seminal vesicle invasion, capsular penetration, and Gleason score. (35 pts)

a-) Use 70% of the data for train data set and use remaining data (30% of data) for test data set (Please use `set.seed(567)`). (5 points)

Ans:

```
Excel.Prostate.data = read.csv("Prostate Cancer.csv")
#View(Excel.Prostate.data)
#str(Excel.Prostate.data)

#Normalized predictors using Min-Max Normalization:
normalize = function(x) {return((x - min(x)) / (max(x) - min(x)))}

Prostate_norm = as.data.frame(lapply(Excel.Prostate.data, normalize))

#To confirm that the normalization worked, we can see that the minimum and
#maximum strength are now zero and one,
summary(Prostate_norm$PSA.level)

##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.00000 0.01887 0.04795 0.08728 0.07820 1.00000

library(caret)

## Loading required package: ggplot2
## Loading required package: lattice
#Splitting the data into training and testing samples:
set.seed(567)

#Do 70/30 split, 70 for Train Data and 30 for Test Data.
DataSplit = createDataPartition( y = Prostate_norm$PSA.level, p = 0.7, list = FALSE )

#Creating test and train data set
TrainData = Prostate_norm[DataSplit, ]
TestData = Prostate_norm[-DataSplit, ]
#View(TrainData)

#Checking if you have 70% test and 30% train
Num_observations = nrow(Prostate_norm)
Num_observations_train = nrow(TrainData)
Num_observations_test = nrow(TestData)

cat("Total # of obs:", Num_observations, "\n")
```

```
## Total # of obs: 97
cat("Total # of train data obs:", Num_observations_train, "\n")

## Total # of train data obs: 70
cat("Total # of test data obs:", Num_observations_test, "\n")

## Total # of test data obs: 27
# Show dimensions
dim(TrainData) # rows and columns

## [1] 70  8
dim(TestData)

## [1] 27  8
```

use the train data set for the questions below

b-) Use neural network (NN) with two hidden layers with 6 neurons and 4 neurons respectively (hidden=c(6,4)) for predicting PSA. Assess the model performance. (10 pts)

Ans: The Neural Network has an error rate of 0.035248 and has taken 318 steps to converge.

The NN model fits the data very well—high correlation and R^2 , along with low RMSE and MAE, indicate strong predictive accuracy with minimal error.

Correlation of 0.9734906

RMSE Rsquared MAE 8.414475 0.947684 5.744736

```
#install.packages("neuralnet")
library(neuralnet)
# WARNING: ### ----- TrainData & TestData is already normalized in 1a -----### w/o normalization, n

# Fit Neural Network with hidden layers 6 and 4
NN_model = neuralnet(PSA.level ~ ., data = TrainData, hidden = c(6,4))

plot(NN_model)

### ----- Model Performance on Training Data ----- ###

NN_model_results = compute(NN_model, TrainData[-c(1)])
# we get rid of the first column, psa.level, response var
predictTrain = NN_model_results$net.result

# Correlation between predicted PSA and actual PSA
cor(predictTrain, TrainData$PSA.level)

##           [,1]
## [1,] 0.9734906

# Scaling Back / going back to original data / Un-normalizing the current data:
unnormalize = function(x) {return((x * (max(Excel.Prostate.data$PSA.level)) -min(Excel.Prostate.data$PSA.level)))}

library(caret)
Predicted_unnormalized = unnormalize(predictTrain)
```

```
ModelTrain1 = data.frame(obs = Excel.Prostate.data$PSA.level[DataSplit], pred=Predicted_unnormalized)
#calculate performance statistics
defaultSummary(ModelTrain1)
```

```
##      RMSE Rsquared      MAE
## 8.414475 0.947684 5.744736
```

c-) Use neural network (NN) with three hidden layers with 8 neurons, 10 neurons and 6 neurons respectively (hidden=c(8,10,6)) for predicting PSA with softplus activation function (softplus = function(x) { log(1 + exp(x)) }). Assess the model performance. (10 pts)

Ans:

The Neural Network has an error rate of 0.012448 and has taken 839 steps to converge using the softplus activation function.

The NN model fits the data very well—high correlation and R^2 , along with low RMSE and MAE, indicate strong predictive accuracy with minimal error.

Correlation of 0.9907177

RMSE Rsquared MAE 5.0238855 0.9815216 3.4511227

```
softplus = function(x) { log(1 + exp(x)) }
```

```
NN_model_2 = neuralnet( PSA.level ~ ., data = TrainData, hidden = c(8,10,6), act.fct = softplus)
```

```
plot(NN_model_2)
```

```
### ----- Model Performance on Training Data ----- ###
```

```
NN_model_results_2 = compute(NN_model_2, TrainData[-c(1)])
```

```
predictTrain_2 = NN_model_results_2$net.result
```

```
# Correlation between predicted PSA and actual PSA
```

```
cor(predictTrain_2, TrainData$PSA.level)
```

```
##      [,1]
```

```
## [1,] 0.9907177
```

```
# Scaling Back / going back to original data / Un-normalizing the current data:
```

```
unnormalize = function(x) {return((x * (max(Excel.Prostate.data$PSA.level)) -min(Excel.Prostate.data$PSA.level)) / (max(Excel.Prostate.data$PSA.level) -min(Excel.Prostate.data$PSA.level)))}
```

```
library(caret)
```

```
Predicted_unnormalized_2 = unnormalize(predictTrain_2)
```

```
ModelTrain2 = data.frame(obs = Excel.Prostate.data$PSA.level[DataSplit], pred=Predicted_unnormalized_2)
```

```
#calculate performance statistics
```

```
defaultSummary(ModelTrain2)
```

```
##      RMSE Rsquared      MAE
```

```
## 5.0238855 0.9815216 3.4511227
```

d-) Based on the performances of the model on the train data set, which model would you choose? (5 pts)

Ans:

Because RMSE and MAE are lower (indicating smaller prediction errors), and both R^2 and correlation are higher (indicating a stronger model fit), the neural network in part (c) outperforms the model in part (b).

Therefore, the model in part (c) is the champion model for the training data.

b: Neuron Network (NN) with two hidden layers with 6 neurons and 4 neurons respectively:

Correlation: 0.9734906

RMSE	Rsquared	MAE
8.414475	0.947684	5.744736

c: Neuron Network (NN) with three hidden layers with 8 neurons, 10 neurons and 6 neurons respectively with Softplus activation:

Correlation: 0.9907177

RMSE	Rsquared	MAE
5.0238855	0.9815216	3.4511227

use the test data set for the questions below

e-) Assess the performance of the models built in parts b and c on the test data set. Which model would you choose? (5 pts)

Ans:

Based on the test set performance, both neural networks show weak generalization (low correlations, high RMSE & MAE). Model 1 performs slightly better than Model 2 on all metrics, so Model 1 (from part b) would be preferred on the test data.

B-» Model 1:

Correlation: 0.3526541

RMSE	Rsquared	MAE
56.2346799	0.1243649	23.6905335

C-» Model 2:

Correlation 0.04802854

RMSE	Rsquared	MAE
56.448645863	0.002306741	24.031396583

```
### ----- B: MODEL 1: hidden = c(6,4), Performance on Training Data ----- ###
```

```
NN_model_Test_results = compute(NN_model, TestData[-c(1)])
predictTest = NN_model_Test_results$net.result
```

```
# Correlation between predicted PSA and actual PSA
cor(predictTest, TestData$PSA.level)
```

```
##           [,1]
## [1,] 0.3526541
```

```
# Scaling Back / going back to original data / Un-normalizing the current data:
```

```
unnormalize = function(x) {return((x * (max(Excel.Prostate.data$PSA.level)) - min(Excel.Prostate.data$PSA.level)) / (max(Excel.Prostate.data$PSA.level) - min(Excel.Prostate.data$PSA.level)))}
```

```
library(caret)
```

```
Predicted_unnormalized_test = unnormalize(predictTest)
```

```
# need to use --DataSplit for the test data
```

```

ModelTest1 = data.frame(obs = Excel.Prostate.data$PSA.level[-DataSplit], pred=Predicted_unnormalized_test1)
#calculate performance statistics
defaultSummary(ModelTest1)

##          RMSE    Rsquared      MAE
## 56.2346799  0.1243649 23.6905335

### ----- C: MODEL 2: hidden = c(8,10,6), Softplus Performance on Training Data ----- ###
cat("\n\n C-->> Model 2: \n\n")

##
##
## C-->> Model 2:
NN_model_Test_results_2 = compute(NN_model_2, TestData[-c(1)])
predictTest_2 = NN_model_Test_results_2$net.result

# Correlation between predicted PSA and actual PSA
cor(predictTest_2, TestData$PSA.level)

##          [,1]
## [1,] 0.04802854

# Scaling Back / going back to original data / Un-normalizing the current data:
unnormailize = function(x) {return((x * (max(Excel.Prostate.data$PSA.level)) -min(Excel.Prostate.data$PSA.level)))}

library(caret)
Predicted_unnormalized_test_2 = unnormailize(predictTest_2)
ModelTest2 = data.frame(obs = Excel.Prostate.data$PSA.level[-DataSplit], pred=Predicted_unnormalized_test_2)
#calculate performance statistics
defaultSummary(ModelTest2)

##          RMSE    Rsquared      MAE
## 56.448645863  0.002306741 24.031396583

```

Problem 2

Use the wbca data(copy and paste the following command into R console:library(faraway);data("pima")

The National Institute of Diabetes and Digestive and Kidney Diseases conducted a study on 768 adult female Pima Indians living near Phoenix. The purpose of the study was to investigate factors related to diabetes. The data may be found in the the dataset pima.

Build a model to predict if a patient shows a positive sign of diabetes (test=1)

The dataset contains the following variables

pregnant: Number of times pregnant

glucose:Plasma glucose concentration at 2 hours in an oral glucose tolerance test

diastolic:Diastolic blood pressure (mm Hg)

triceps:Triceps skin fold thickness (mm)

insulin:2-Hour serum insulin (mu U/ml)

bmi:Body mass index (weight in kg/(height in metres squared))

diabetes:Diabetes pedigree function

age: Age (years)

test: test whether the patient shows signs of diabetes (coded 0 if negative, 1 if positive) (65 pts, 5 points each)

a-) Use 70% of the data for train data set and use remaining data (30% of data) for test data set (Please use set.seed(567)). (5 points)

Ans:

```
library(faraway)

##
## Attaching package: 'faraway'
## The following object is masked from 'package:lattice':
##
##      melanoma

data("pima")
#View(pima)
#help("pima")
str(pima)

## 'data.frame':    768 obs. of  9 variables:
## $ pregnant : int  6 1 8 1 0 5 3 10 2 8 ...
## $ glucose   : int  148 85 183 89 137 116 78 115 197 125 ...
## $ diastolic : int  72 66 64 66 40 74 50 0 70 96 ...
## $ triceps   : int  35 29 0 23 35 0 32 0 45 0 ...
## $ insulin   : int  0 0 0 94 168 0 88 0 543 0 ...
## $ bmi       : num  33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 0 ...
## $ diabetes  : num  0.627 0.351 0.672 0.167 2.288 ...
## $ age       : int  50 31 32 21 33 30 26 29 53 54 ...
## $ test      : int  1 0 1 0 1 0 1 0 1 1 ...

#head(pima)

library(caret)

#Splitting the data into training and testing samples:
set.seed(567)

#Do 70/30 split, 70 for Train Data and 30 for Test Data.
DataSplit = createDataPartition( y = pima$test, p = 0.7, list = FALSE )

#Creating test and train data set
PimaTrainData = pima[DataSplit, ]
PimaTestData = pima[-DataSplit, ]
#View(TrainData)

#Checking if you have 70% test and 30% train
Num_observations = nrow(pima)
Num_observations_train = nrow(PimaTrainData)
Num_observations_test = nrow(PimaTestData)

cat("Total # of obs:", Num_observations, "\n")

## Total # of obs: 768
```

```
cat("Total # of train data obs:", Num_observations_train, "\n")
```

```
## Total # of train data obs: 538
```

```
cat("Total # of test data obs:", Num_observations_test, "\n")
```

```
## Total # of test data obs: 230
```

```
# Show dimensions
```

```
dim(PimaTrainData) # rows and columns
```

```
## [1] 538  9
```

```
dim(PimaTestData)
```

```
## [1] 230  9
```

use the train data set for the questions below

b-) Fit a binary logistic regression with `test` as the response and the variables listed above as predictors on the training data set. Report the residual deviance and associated degrees of freedom. Can this information be used to determine whether this model fits the data? Explain. (5 points)

Ans:

Because the response variable “test” indicates whether a patient shows signs of diabetes (1 = positive, 0 = negative), logistic regression is an appropriate modeling choice.

Residual deviance: 502.99 on 529 degrees of freedom

In logistic regression, the residual deviance plays the same role as the residual sum of squares in linear regression: it measures how far the fitted model is from a perfect (saturated) model. A smaller deviance indicates a better fit.

If the model fits well, the residual deviance should be close to its degrees of freedom → Residual Deviance is approx equal to Degrees of Freedom.

Because 502.99 is reasonably close to 529, this does not suggest major lack of fit. However, this single number cannot guarantee that the model fits well everywhere.

Therefore, while the deviance does not indicate a clear lack of fit, we cannot conclude that the model is adequate without additional diagnostics—such as Hosmer–Lemeshow tests, residual plots, etc.

```
# Fit full logistic regression on the training data
```

```
Logistic_full_regression = glm(test ~ ., data=PimaTrainData, family = binomial)
```

```
summary(Logistic_full_regression)
```

```
##
```

```
## Call:
```

```
## glm(formula = test ~ ., family = binomial, data = PimaTrainData)
```

```
##
```

```
## Coefficients:
```

```
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept) -8.3443997  0.8601241  -9.701  < 2e-16 ***
## pregnant    0.1283170  0.0385175   3.331 0.000864 ***
## glucose     0.0346747  0.0043245   8.018 1.07e-15 ***
## diastolic   -0.0177177  0.0064913  -2.729 0.006344 **
## triceps     -0.0023394  0.0083050  -0.282 0.778182
## insulin     -0.0009873  0.0010400  -0.949 0.342456
## bmi         0.0910241  0.0181276   5.021 5.13e-07 ***
```

```
## diabetes      0.9389995  0.3526551   2.663 0.007753 **
## age           0.0252921  0.0116411   2.173 0.029806 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 703.45  on 537  degrees of freedom
## Residual deviance: 502.99  on 529  degrees of freedom
## AIC: 520.99
##
## Number of Fisher Scoring iterations: 5
```

c-) Use AIC as the criterion to determine the best subset of variables (use the **step** function). (5 points)

Ans: All of the predictor p-values are less than 0.05, so all the predictor variables are statistically significant, and nothing needs to be removed.

Using the step() function with AIC as the selection criterion, the best model chosen is:

test ~ pregnant + glucose + diastolic + bmi + diabetes + age

The AIC value for this model is 518.44

Residual deviance: 504.44 on 531 degrees of freedom

Hosmer-Lemeshow Goodness of Fit Test:

Null Hypothesis: If $P > 0.05$: The logistic regression model fits the data well.

Alternative Hypothesis: If $P \leq 0.05$ The logistic regression model does not fits the data well.

p-value = 0.1881 > 0.05 so we fail to reject the null hypothesis.

This suggests that the model fits the data adequately.

```
logistic_stepwsie = step(Logistic_full_regression, direction = "both")
```

```
## Start:  AIC=520.99
## test ~ pregnant + glucose + diastolic + triceps + insulin + bmi +
##      diabetes + age
##
##              Df Deviance    AIC
## - triceps     1   503.07  519.07
## - insulin     1   503.89  519.89
## <none>         502.99  520.99
## - age         1   507.75  523.75
## - diabetes    1   510.33  526.33
## - diastolic   1   510.74  526.74
## - pregnant    1   514.57  530.57
## - bmi         1   532.34  548.34
## - glucose     1   585.99  601.99
##
## Step:  AIC=519.07
## test ~ pregnant + glucose + diastolic + insulin + bmi + diabetes +
##      age
##
##              Df Deviance    AIC
## - insulin     1   504.44  518.44
## <none>         503.07  519.07
```



```

## + triceps      1    502.99 520.99
## - age          1    508.08 522.08
## - diabetes     1    510.33 524.33
## - diastolic    1    511.43 525.43
## - pregnant     1    514.65 528.65
## - bmi          1    533.98 547.98
## - glucose      1    588.11 602.11
##
## Step: AIC=518.44
## test ~ pregnant + glucose + diastolic + bmi + diabetes + age
##
##           Df Deviance    AIC
## <none>          504.44 518.44
## + insulin      1    503.07 519.07
## + triceps      1    503.89 519.89
## - age          1    510.19 522.19
## - diabetes     1    511.06 523.06
## - diastolic    1    513.30 525.30
## - pregnant     1    516.47 528.47
## - bmi          1    534.37 546.37
## - glucose      1    591.89 603.89
summary(logistic_stepwsie)

##
## Call:
## glm(formula = test ~ pregnant + glucose + diastolic + bmi + diabetes +
##      age, family = binomial, data = PimaTrainData)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -8.191762   0.844428  -9.701 < 2e-16 ***
## pregnant     0.130430   0.038449   3.392 0.000693 ***
## glucose      0.033230   0.004026   8.253 < 2e-16 ***
## diastolic    -0.018424   0.006314  -2.918 0.003522 **
## bmi          0.087568   0.017272   5.070 3.98e-07 ***
## diabetes     0.878366   0.346454   2.535 0.011235 *
## age          0.027398   0.011465   2.390 0.016864 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 703.45  on 537  degrees of freedom
## Residual deviance: 504.44  on 531  degrees of freedom
## AIC: 518.44
##
## Number of Fisher Scoring iterations: 5
### Hosmer-Lemeshow Goodness of Fit Test
library(ResourceSelection)

## ResourceSelection 0.3-6    2023-06-27
hoslem.test(logistic_stepwsie$y, fitted(logistic_stepwsie), g=10)

```

```
##
## Hosmer and Lemeshow goodness of fit (GOF) test
##
## data: logistic_stepwsie$y, fitted(logistic_stepwsie)
## X-squared = 11.248, df = 8, p-value = 0.1881
```

```
#View(PimaTrainData)
```

d-) Using the reduced logistic regression model selected in part c, suppose that a case is classified as positive if $p > 0.5$ and negative if $p < 0.5$. Compute the confusion matrix and the accuracy rate when this method is applied to the training data. (5 points)

Ans: Definitions: Recall (Sensitivity): $TP / (TP + FN)$ Specificity: $TN / (TN + FP)$

Recall = $121 / (121 + 73) = 0.6237$ The model struggles to identify positives

Specificity = $300 / (300 + 44) = 0.87209$ The model is very good at identifying negatives

From the confusion matrix, the model achieved an accuracy of 78.25%, with a Recall (sensitivity) of 62.37% and a specificity of 87.21%. The model performs moderately well on the training data. The accuracy is significantly higher than the No Information Rate (63.94%), with a very small p-value ($4.7e-13 < 0.05$), indicating that the model outperforms random guessing.

Overall, the logistic regression model is a decent/moderate fit for the data set.

```
# class labels
#actual_labels = factor(PimaTrainData$test, levels = c("0", "1"))
str(PimaTrainData)
```

```
## 'data.frame': 538 obs. of 9 variables:
## $ pregnant : int 1 8 1 5 3 2 8 1 7 0 ...
## $ glucose : int 85 183 89 116 78 197 125 189 100 118 ...
## $ diastolic: int 66 64 66 74 50 70 96 60 0 84 ...
## $ triceps : int 29 0 23 0 32 45 0 23 0 47 ...
## $ insulin : int 0 0 94 0 88 543 0 846 0 230 ...
## $ bmi : num 26.6 23.3 28.1 25.6 31 30.5 0 30.1 30 45.8 ...
## $ diabetes : num 0.351 0.672 0.167 0.201 0.248 0.158 0.232 0.398 0.484 0.551 ...
## $ age : int 31 32 21 30 26 53 54 59 32 31 ...
## $ test : int 0 1 0 0 1 1 1 1 1 1 ...
```

```
#View(PimaTrainData)
```

```
# It returns probabilities that the observations are "positive"/
# making predictions:
```

```
predicted_probabilities = predict(logistic_stepwsie, PimaTrainData, type = "response")
```

```
# lets use .5 as a cutoff and Create confusion matrix
```

```
predictions = ifelse(predicted_probabilities > 0.5, 1, 0)
```

```
# Confusion Matrix:
```

```
confusion_matrix = confusionMatrix(data = as.factor(predictions),
reference = as.factor(PimaTrainData$test), mode = "prec_recall", positive = "1")
confusion_matrix
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction  0    1
##           0 300  73
##           1  44 121
```

```
##
##           Accuracy : 0.7825
##           95% CI : (0.7452, 0.8167)
##      No Information Rate : 0.6394
##      P-Value [Acc > NIR] : 4.707e-13
##
##           Kappa : 0.5125
##
## Mcnemar's Test P-Value : 0.009637
##
##           Precision : 0.7333
##           Recall : 0.6237
##           F1 : 0.6741
##           Prevalence : 0.3606
##      Detection Rate : 0.2249
##      Detection Prevalence : 0.3067
##      Balanced Accuracy : 0.7479
##
##      'Positive' Class : 1
##
```

e-) Develop a classification tree for predicting a positive test result using the training data. Compute the confusion matrix and comment on the model accuracy. (10 pts)

Ans:

From the CV plot, the lowest deviance occurs at tree size of 6, indicating the optimal level of complexity for pruning.

Below are the performance results:

Pruned Tree: Accuracy: 0.803 Sensitivity (Recall): 0.5722 Specificity: 0.9331 P-Value [Acc > NIR] : < 2.2e-16

Unpruned Tree: Accuracy : 0.816

P-Value [Acc > NIR] : < 2e-16

Sensitivity : 0.7268

Specificity : 0.8663

The full tree achieves higher accuracy and much higher sensitivity, meaning it identifies positive diabetes cases more effectively.

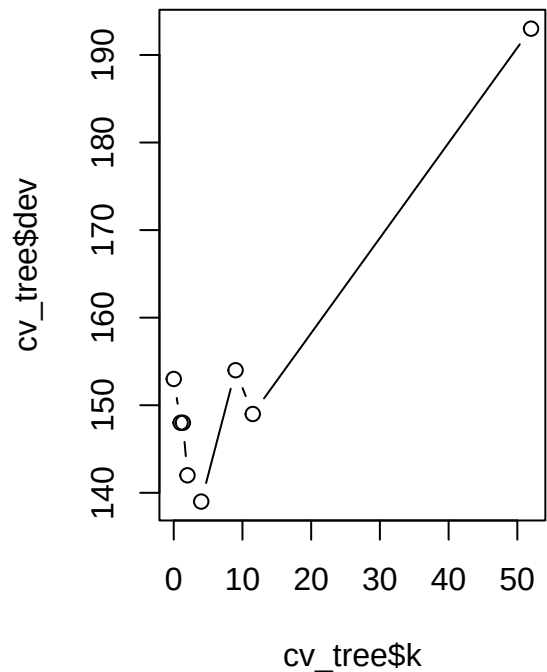
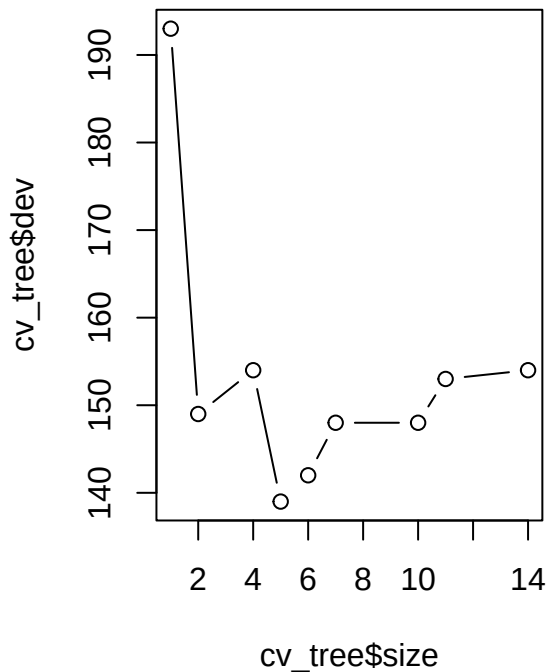
Pruning reduces the model's complexity, but in this case it reduces predictive performance on the training data. Since the full tree has higher accuracy and better balance between sensitivity and specificity, pruning is not needed here, and the unpruned tree is the better model on the training data.

```
library(tree)
library(caret)
set.seed(567)

PimaTrainData$test = factor(PimaTrainData$test, levels = c("0", "1"))
# Full Tree Model:
tree_full_model = tree(test ~ ., data = PimaTrainData)

# Performing Cv to find the optimal tree size:
cv_tree = cv.tree(tree_full_model, FUN = prune.misclass)

par(mfrow = c(1, 2))
plot(cv_tree$size, cv_tree$dev, type = "b")
plot(cv_tree$k, cv_tree$dev, type = "b")
```



```
# Note: Size 6 has the lowest CV error (lowest point on the plot)

# Pruning tree with 6 terminal nodes:
# note: best tree size is found from looking at the left most node graph
# and playign around.
prune_tree_model = prune.misclass(tree_full_model, best = 6)

# Plot pruned tree:
plot(prune_tree_model)
text(prune_tree_model, pretty = 0)

# Predict classes on the train data:
Pruned_Predict_Train = predict(prune_tree_model, PimaTrainData, type = "class")

#Confusion matrix for the Pruned Tree:
PrunedTreeconfusionMatrix = confusionMatrix(data = as.factor(Pruned_Predict_Train),
reference = as.factor(PimaTrainData$test), positive = "1")
cat("\n\n Pruned Tree: \n\n")
```

```
##
```

```
##
```

```
## Pruned Tree:
```

```
PrunedTreeconfusionMatrix
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction  0    1
```

```
##           0 321  83
```

```
##           1  23 111
```

```
##
```

```
##           Accuracy : 0.803
```

```
##           95% CI : (0.7668, 0.8358)
```

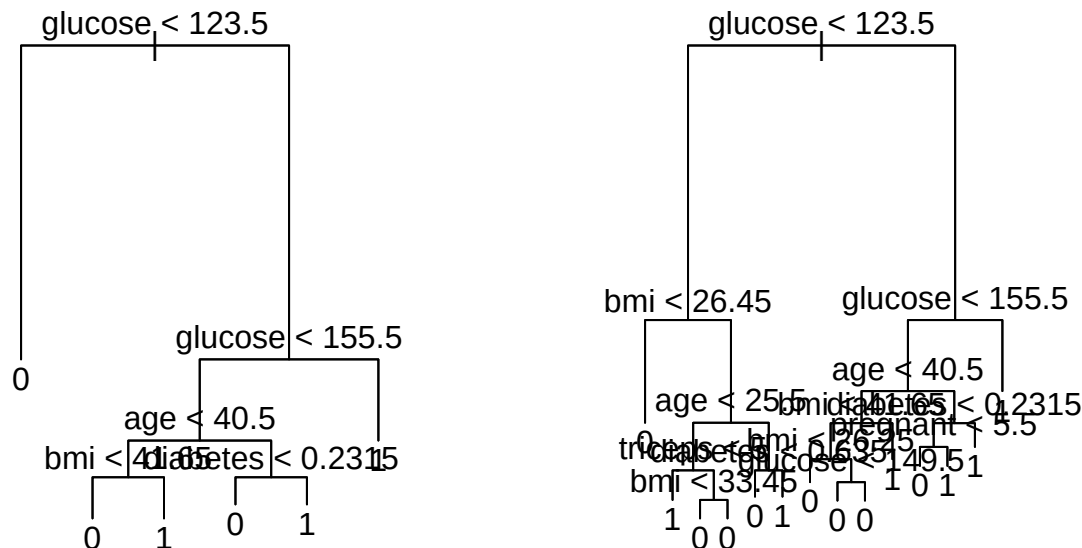
```
##      No Information Rate : 0.6394
##      P-Value [Acc > NIR] : < 2.2e-16
##
##      Kappa : 0.5418
##
##      Mcnemar's Test P-Value : 1.001e-08
##
##      Sensitivity : 0.5722
##      Specificity : 0.9331
##      Pos Pred Value : 0.8284
##      Neg Pred Value : 0.7946
##      Prevalence : 0.3606
##      Detection Rate : 0.2063
##      Detection Prevalence : 0.2491
##      Balanced Accuracy : 0.7527
##
##      'Positive' Class : 1
##
```

```
##### ---- For Full TREE Model ----- #####
```

```
###Plotting the Full tree:
```

```
plot(tree_full_model)
```

```
text(tree_full_model, pretty = 0)
```



```
# Predict classes on the train data:
```

```
Predict.Train = predict(tree_full_model, PimaTrainData, type = "class")
```

```
#Confusion matrix:
```

```
TreeconfusionMatrix = confusionMatrix(data = as.factor(Predict.Train),
```

```
reference = as.factor(PimaTrainData$test), positive = "1")
```

```
cat("\n\n Full Tree Model: \n\n")
```

```
##
```

```
##
```

```
## Full Tree Model:
```

```
TreeconfusionMatrix
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction   0   1
##           0 298  53
##           1  46 141
##
##           Accuracy : 0.816
##           95% CI : (0.7806, 0.8478)
##           No Information Rate : 0.6394
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.5978
##
## Mcnemar's Test P-Value : 0.5465
##
##           Sensitivity : 0.7268
##           Specificity : 0.8663
##           Pos Pred Value : 0.7540
##           Neg Pred Value : 0.8490
##           Prevalence : 0.3606
##           Detection Rate : 0.2621
##           Detection Prevalence : 0.3476
##           Balanced Accuracy : 0.7965
##
##           'Positive' Class : 1
##
```

f-) Use a neural network (NN) with two hidden layers with 6 and 4 neurons respectively (for example, `hidden = c(6, 4)`) for predicting a positive test result. Suppose that a case is classified as positive if $p > 0.5$ and negative if $p < 0.5$. Compute the confusion matrix and comment on the model performance. (10 pts)

Ans: Accuracy : 0.6394
P-Value [Acc > NIR] : 0.5196
Sensitivity : 1.0000
Specificity : 0.0000

The Neural Network has an error rate of 97.014009 and has taken 19 steps to converge. The neural network does not perform well and is not reliable for classification. This can be seen since the accuracy rate is about 64%. It fails to distinguish between positive and negative test outcomes. Based on the results alone, NN would not be for this kind of data.

```
# install.packages("neuralnet")
# install.packages("caret")
library(neuralnet)
library(caret)
set.seed(567)

### -----
### TRAINING DATA BASED NORMALIZATION PARAMETERS
### -----

#parameters (center/scale) on TRAINING data ONLY (Predictors 1-8)
params = preProcess(PimaTrainData[, 1:8], method=c("center", "scale"))

# Applyign standardization to train and test predictors
PimaTrainData[, 1:8] = predict(params, PimaTrainData[, 1:8])
```

```

PimaTestData[, 1:8] = predict(params, PimaTestData[, 1:8])

# Keep target variable ('test') as numeric 0/1
PimaTrainData$test = as.numeric(PimaTrainData$test)
PimaTestData$test = as.numeric(PimaTestData$test)
#View(PimaTestData)
### -----
### Fit Neural Network Model with two hidden layers 6 and 4
### -----

NN_modelPima = neuralnet(
  test ~ .,
  data = PimaTrainData,
  hidden = c(6,4),
  linear.output = FALSE,
  stepmax = 1e6,
  threshold = 0.02 )

# Plot NN
plot(NN_modelPima)

### -----
### Predictions on Train Data
### -----

# Training predictions
nn_train_results = compute(NN_modelPima, PimaTrainData[, 1:8])
nn_train_prob = nn_train_results$net.result

# Convert to class labels
nn_train_class = ifelse(nn_train_prob > 0.5, 1, 0)

### -----
### Confusion Matrix (MODEL PERFORMANCE)
### -----

PimaConfusionMatrix_2f = confusionMatrix(
  data = as.factor(nn_train_class),
  reference = as.factor(PimaTrainData$test),
  positive = "1"
)

## Warning in confusionMatrix.default(data = as.factor(nn_train_class), reference
## = as.factor(PimaTrainData$test), : Levels are not in the same order for
## reference and data. Refactoring data to match.

PimaConfusionMatrix_2f

## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1    2
##           1 344 194

```

```

##          2    0    0
##
##          Accuracy : 0.6394
##          95% CI : (0.5972, 0.68)
##    No Information Rate : 0.6394
##    P-Value [Acc > NIR] : 0.5196
##
##          Kappa : 0
##
##    McNemar's Test P-Value : <2e-16
##
##          Sensitivity : 1.0000
##          Specificity : 0.0000
##    Pos Pred Value : 0.6394
##    Neg Pred Value :    NaN
##          Prevalence : 0.6394
##    Detection Rate : 0.6394
##    Detection Prevalence : 1.0000
##    Balanced Accuracy : 0.5000
##
##    'Positive' Class : 1
##

```

g-) Based on the performance of the models on the training data set, which model would you choose? (5 pts)

Ans: Logistic Regression Model results: From the confusion matrix, the model achieved an accuracy of 78.25%, with a Recall (sensitivity) of 62.37% and a specificity of 87.21%. The model performs moderately well on the training data. The accuracy is significantly higher than the No Information Rate (63.94%), with a very small p-value ($4.7\text{e-}13 < 0.05$), indicating that the model outperforms random guessing.

Classification Tree results: Unpruned Tree: Accuracy : 0.816

P-Value [Acc > NIR] : < 2e-16

Sensitivity : 0.7268

Specificity : 0.8663

NN results: Accuracy : 0.6394

P-Value [Acc > NIR] : 0.5196

Sensitivity : 1.0000

Specificity : 0.0000

Champion: The Classification Tree is the best-performing model because it achieves the highest accuracy, maintains strong sensitivity and specificity. It provides the most reliable overall performance on the training data.

use the test data set for the questions below

h-) Compute the confusion matrix of the logistic regression model built on the training data using $p = 0.5$ as a cutoff. Is the model performance stable? (5 pts)

Ans: Test Data: Accuracy : 0.6783

No Information Rate : 0.6783

P-Value [Acc > NIR] : 0.5314

Sensitivity : 0.0000

Specificity : 1.0000

Train Data: Accuracy : 78.25%

No Information Rate : 63.94%

P-Value [Acc > NIR] : $4.707\text{e-}13$

Sensitivity : 62.37%
Specificity : 87.21%

The model performance is not stable. It performs much worse on the test data, accuracy drops, and sensitivity collapses to 0, meaning it fails to detect any positive cases. This shows poor generalization and likely over fitting on the training set.

```
#From part 2c:
#####
# Fit stepwise reduced logistic regression on the training data
#logistic_stepwsie = step(Logistic_full_regression, direction = "both")

#####

# Predict probabilities on Test Data:

logistic_prob = predict(logistic_stepwsie, newdata = PimaTestData, type = "response")
View(PimaTestData)
str(PimaTestData)
```

```
## 'data.frame': 230 obs. of 9 variables:
## $ pregnant : num 0.6492 -1.1298 1.8352 0.0562 1.8352 ...
## $ glucose : num 0.821 0.489 -0.176 -0.327 1.426 ...
## $ diastolic: num 0.12 -1.577 -3.697 1.18 0.226 ...
## $ triceps : num 0.881 0.881 -1.317 -1.317 -1.317 ...
## $ insulin : num -0.704 0.697 -0.704 -0.704 -0.704 ...
## $ bmi : num 0.195 1.412 0.413 0.708 0.759 ...
## $ diabetes : num 0.446 5.292 -0.992 -0.826 0.184 ...
## $ age : num 1.4295 -0.0387 -0.3841 -0.2978 0.0477 ...
## $ test : num 1 1 0 0 1 0 1 1 1 ...
```

converting probabilities into 0 or 1 predictions using p = 0.5 as a cutoff:

```
logistic_test_class = ifelse(logistic_prob > 0.5, 1, 0)
# View(logistic_test_class)
```

Actual outcome from the Test Data:
actual_test_class = PimaTestData\$test

```
#####
# Confusion Matrix:
#####
library(caret)
```

```
### data = predicted, and reference = actual ###
LogisticConfusionMatrix = confusionMatrix(data = as.factor(logistic_test_class),
                                           reference = as.factor(actual_test_class) , positive = "1")
```

```
## Warning in confusionMatrix.default(data = as.factor(logistic_test_class), :
## Levels are not in the same order for reference and data. Refactoring data to
## match.
```

LogisticConfusionMatrix

```
## Confusion Matrix and Statistics
##
##           Reference
```

```
## Prediction    0    1
##              0 156  74
##              1   0   0
##
##              Accuracy : 0.6783
##              95% CI  : (0.6137, 0.7382)
##      No Information Rate : 0.6783
##      P-Value [Acc > NIR] : 0.5314
##
##              Kappa : 0
##
##  McNemar's Test P-Value : <2e-16
##
##      Sensitivity : 0.0000
##      Specificity : 1.0000
##      Pos Pred Value :    NaN
##      Neg Pred Value : 0.6783
##      Prevalence : 0.3217
##      Detection Rate : 0.0000
##      Detection Prevalence : 0.0000
##      Balanced Accuracy : 0.5000
##
##      'Positive' Class : 1
##
```

i-) Compute the confusion matrix of the classification tree built in part e. Is the model performance stable? (5 pts)

Ans: Test Data: Accuracy : 0.6783 No Information Rate : 0.6783 P-Value [Acc > NIR] : 0.5314 Sensitivity : 0.0000 Specificity : 1.0000

Train Data: Unpruned Tree: Accuracy : 0.816 No Information Rate : 0.6394
P-Value [Acc > NIR] : < 2e-16
Sensitivity : 0.7268
Specificity : 0.8663

The model performance is not stable. It performs much worse on the test data, accuracy drops, and sensitivity collapses to 0, meaning it fails to detect any positive cases. This shows poor generalization and likely over fitting on the training set.

```
## # Full Tree Model:
## tree_full_model = tree(test ~ ., data = PimaTrainData)

PimaTestData$test = as.factor(PimaTestData$test)

# Predict class labels on the TEST data:
Predict.Test = predict(tree_full_model, PimaTestData, type = "class")
#summary(Predict.Train)

#Confusion matrix:
# Error: `data` and `reference` should be factors with the same levels.
# to fix this error I have used as.factor so that R considers them as categories and not numbers.
# as.factor() tells R to treat a variable as categorical like (benign or malignant),
# not numbers (like 0 and 1).

TestConfusionMatrix = confusionMatrix(data = as.factor (Predict.Test), reference = as.factor (PimaTestD
```

TestConfusionMatrix

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 156  74
##           1   0   0
##
##           Accuracy : 0.6783
##           95% CI : (0.6137, 0.7382)
##       No Information Rate : 0.6783
##       P-Value [Acc > NIR] : 0.5314
##
##           Kappa : 0
##
##  Mcnemar's Test P-Value : <2e-16
##
##           Sensitivity : 0.0000
##           Specificity : 1.0000
##       Pos Pred Value :    NaN
##       Neg Pred Value : 0.6783
##           Prevalence : 0.3217
##       Detection Rate : 0.0000
##   Detection Prevalence : 0.0000
##       Balanced Accuracy : 0.5000
##
##       'Positive' Class : 1
##
```

j-) Compute the confusion matrix of the neural network (NN) model built in part f using $p = 0.5$ as a cutoff. Is the model performance stable? (5 pts)

Ans: Test: Accuracy : 0.3217 No Information Rate : 0.6783 P-Value [Acc > NIR] : 1 Sensitivity : 1.0000 Specificity : 0.0000

Train from 2f: Accuracy : 0.6394
P-Value [Acc > NIR] : 0.5196
Sensitivity : 1.0000
Specificity : 0.0000

The neural network model's performance is not stable. Although it achieves perfect sensitivity on both the training and test sets (detecting all positive cases), its accuracy drops substantially when applied to the test data, and its specificity falls to zero. This means the model predicts almost everything as positive. The large gap between the training accuracy (about 0.64) and the much lower test accuracy (about 0.32) indicates that the model is overfitting the training data and failing to generalize to unseen observations. Overall, the unstable accuracy, unchanged sensitivity, and severe loss in specificity confirm that the neural network does not perform reliably on new data.

```
library(neuralnet)
library(caret)
set.seed(567)
```

```
### -----
### Predictions of probabilities on Test Data
### -----
```

```

NN_test_results = compute(NN_modelPima, PimaTestData[,1:8])
prediction_test_probabilities = NN_test_results$net.result

# Convert to class labels, lets use .5 as a cutoff and Create confusion matrix
prediction_test_class = ifelse(prediction_test_probabilities > 0.5, 1, 0)

### -----
### Confusion Matrix (MODEL PERFORMANCE) on TEST DATA:
### -----

PimaConfusionMatrix_2j = confusionMatrix(
  data = as.factor(prediction_test_class),
  reference = as.factor(PimaTestData$test),
  positive = "1")

## Warning in confusionMatrix.default(data = as.factor(prediction_test_class), :
## Levels are not in the same order for reference and data. Refactoring data to
## match.

PimaConfusionMatrix_2j

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0    0    0
##           1 156   74
##
##           Accuracy : 0.3217
##           95% CI : (0.2618, 0.3863)
##       No Information Rate : 0.6783
##       P-Value [Acc > NIR] : 1
##
##           Kappa : 0
##
##  McNemar's Test P-Value : <2e-16
##
##           Sensitivity : 1.0000
##           Specificity : 0.0000
##       Pos Pred Value : 0.3217
##       Neg Pred Value :    NaN
##           Prevalence : 0.3217
##       Detection Rate : 0.3217
##  Detection Prevalence : 1.0000
##       Balanced Accuracy : 0.5000
##
##       'Positive' Class : 1
##

```

k-) Based on the performance on the test data set, which model would you choose? (5 pts)

Ans: Logistic Regression Model: Test Data: Accuracy : 0.6783

No Information Rate : 0.6783

P-Value [Acc > NIR] : 0.5314

Sensitivity : 0.0000

Specificity : 1.0000

Classification Tree: Test Data: Accuracy : 0.6783 No Information Rate : 0.6783 P-Value [Acc > NIR] : 0.5314
Sensitivity : 0.0000 Specificity : 1.0000

NN: Test: Accuracy : 0.3217 No Information Rate : 0.6783 P-Value [Acc > NIR] : 1 Sensitivity : 1.0000
Specificity : 0.0000

Based on the performance on the test data set, the logistic regression model and the classification tree model clearly outperform the neural network. Both logistic regression and the tree achieve an accuracy of 0.6783 or about 68%, which matches the No Information Rate, while the neural network performs extremely poorly with an accuracy of 0.3217 or about 32% and a specificity of 0, indicating severe overfitting.

Although logistic regression and the tree have similar accuracy and sensitivity/specificity profiles, logistic regression is typically more stable and less prone to overfitting, especially with small datasets like Pima. Therefore, I would choose the logistic regression model as the best-performing and most reliable model on the test data. The second best model would be the classification model due to the aforementioned reasons.